

Student Education Services Architecture Design

<https://gemini.google.com/app/3dff952ee4697068>

User prompt: make proper development architecture flow documentation idea about student education services where student will goes admission in different courses the database will be mongoDB and collection schema will be based on given mysql database as reference so that mongodb database will created based on sql scheme create project ,database will be mongoDB, React with bootstrap css, Node Js and create Smart Contract for certificate using Blockchain with help of ganache and metamask tools and all blocks information will saved in mongoDB

Response: This development architecture flow outlines a modern, full-stack Web3 application tailored for student education services. Designed by Saikat Sengupta, this document integrates a MERN stack (MongoDB, Express, React, Node.js) with an Ethereum-based blockchain layer for immutable certificate issuance.

Technology Stack Overview

- **Frontend:** React.js integrated with Bootstrap CSS for a responsive admission portal and student dashboard.
- **Backend:** Node.js using Express to handle RESTful APIs, student authentication, and database routing.
- **Database:** MongoDB to store all relational entities. Mongoose ORM will handle the translation of the SQL schema into NoSQL document models.
- **Blockchain Integration:** Ethereum smart contracts written in Solidity for generating and verifying certificates.
- **Web3 Tooling:** Ganache for the local Ethereum blockchain network and MetaMask for signing transactions from the application.

Database Migration: MySQL to MongoDB Schema

We will map the relational tables from your `studentPortal.sql` file into MongoDB collections. The schema is structured to ensure that blockchain transaction data syncs seamlessly back into MongoDB.

1. Course Collection

This collection will store the initial seeded course data (e.g., Computer Science Diploma, BCA, Data Science).

- `courseid`: String (Primary Identifier)
- `coursename`: String (Required)
- `course_type`: String (Required)
- `course_abbr`: String (Required)
- `price`: Number (Required, Default: 0.00)
- `duration`: Number (Required)

2. Student Collection

- `sid`: String (Primary Identifier)
- `courseid`: String (Reference to Course)
- `fname & lname`: String (Required)
- `username`: String (Required, Unique)
- `password & passKey`: String (Required)
- `flag`: Number (Default: 1)
- `email`: String (Required, Unique)
- `phone, city, photo`: String
- `dob`: Date
- `created_at`: Date (Default: Current Timestamp)

3. Examination & Result Collections

- **Examination:** Includes `examid`, `sid`, `courseid`, `rsultid`, `exam_date`, `created_on`, `update_on`, and `update_by`.
- **Result:** Includes `rsultid`, `examid`, `sid`, `no_of_sub`, `total_marks`, `rst_created_on`, and `rst_update_on`.

4. Certificate Collection (With Blockchain Sync)

This collection maps the base SQL schema while adding Web3 data to track the Ethereum network block information.

- `crtid`: String (Primary Identifier)

- `sid`: String (Reference to Student)
- `fullname`: String (Required)
- `examid & coursename`: String
- `score & grade`: String
- `crt_generate_on & crt_update_on`: Date
- `blockHash`: String (New - Syncs from Ganache)
- `transactionHash`: String (New - Syncs from MetaMask)

Core Operational Flows

The following sequences illustrate how the React frontend, Node backend, and Ethereum blockchain communicate during critical operations.

1 Student Admission Flow

Frontend to Backend

The student selects a course on the React frontend and submits their registration details. The React application sends a POST request to the Node.js backend API.

2 Database Enrollment

Backend to MongoDB

Node.js validates the admission payload and creates a new document in the MongoDB `Student` collection, referencing the chosen `courseid`.

3 Examination and Grading

Administrative Action

Upon course completion, an admin inputs the student's grades. Node.js updates the `Examination` and `Result` collections in MongoDB with the final scores.

1 Initiate Certificate Generation

Smart Contract Trigger

Once the result is finalized, the backend triggers the Ethereum smart contract via Web3.js/Ethers.js.

2 Sign Transaction

MetaMask Integration

The authorized issuer (e.g., the university admin) signs the transaction using their MetaMask wallet. The transaction contains the student's ID, course name, and grade.

3 Mint on the Blockchain

Ganache Network

The smart contract executes on the local Ganache network, permanently storing the certificate data on the decentralized ledger.

4 Sync Block Data to MongoDB

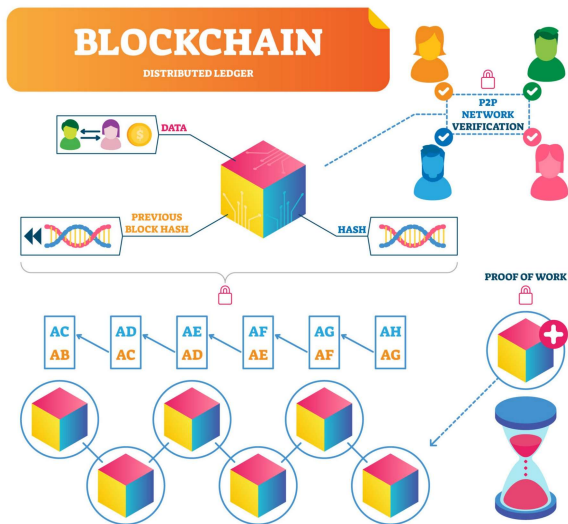
Immutable Record Keeping

Ganache returns a transaction receipt containing the block number and transaction hash. Node.js takes these cryptographic details and saves them into the MongoDB `Certificate` collection for fast front-end querying.

User prompt: need both of them , certificate smart contract will contain blockchain block so that each block contain previous hash and current hash , transaction details and body contain data of student id, name, certificate id, marks score, grade, course name, phone and email all of them saved in database and

Response: Here is the complete implementation for both the Ethereum smart contract and the MongoDB Mongoose schemas.

While Ethereum's native architecture automatically handles the linking of previous and current block hashes at the protocol level, building these fields directly into our smart contract's data structure is an excellent way to explicitly verify the integrity of the certificate data at the application layer.



1. Ethereum Smart Contract (Solidity)

This contract defines a struct that captures the exact payload you requested. It dynamically generates a `currentHash` based on the student's data and the `previousHash`, ensuring a tightly coupled, tamper-proof record.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

contract StudentCertificatePortal {

    // The core data structure requested for the blockchain
    struct CertificateBlock {
        string previousHash;
        string currentHash;
        string transactionDetails;
        string studentId;
        string name;
        string certificateId;
        string marksScore;
        string grade;
        string courseName;
        string phone;
        string email;
        uint256 timestamp;
    }

    // Map certificate IDs to their blockchain record
    mapping(string => CertificateBlock) public certificates;

    // Track the hash of the last generated certificate to link the chain
    string public lastBlockHash;

    event CertificateMinted(string certificateId, string currentHash, uint256 timestamp);

    constructor() {
        // Initialize the genesis hash
        lastBlockHash = "GENESIS_BLOCK_00000000000000000000000000000000";
    }

    // Function to generate and store a new certificate block
    function mintCertificate(
        string memory _transactionDetails,
        string memory _studentId,
        string memory _name,
        string memory _certificateId,
        string memory _marksScore,
        string memory _grade,
        string memory _courseName,
        string memory _phone,
        string memory _email
    ) public {

        // Generate a pseudo-hash for the current block data
        bytes32 generatedHash = keccak256(
```

```

        abi.encodePacked(
            lastBlockHash,
            _studentId,
            _certificateId,
            _marksScore,
            _grade,
            block.timestamp
        )
    );

    string memory currentHashStr = toHexString(uint256(generatedHash), 32);

    // Create the new certificate block
    certificates[_certificateId] = CertificateBlock({
        previousHash: lastBlockHash,
        currentHash: currentHashStr,
        transactionDetails: _transactionDetails,
        studentId: _studentId,
        name: _name,
        certificateId: _certificateId,
        marksScore: _marksScore,
        grade: _grade,
        courseName: _courseName,
        phone: _phone,
        email: _email,
        timestamp: block.timestamp
    });

    // Update the last hash for the next block
    lastBlockHash = currentHashStr;

    emit CertificateMinted(_certificateId, currentHashStr, block.timestamp);
}

// Helper function to convert bytes32 to string
function toHexString(uint256 value, uint256 length) internal pure returns (string memory) {
    bytes memory buffer = new bytes(2 * length + 2);
    buffer[0] = "0";
    buffer[1] = "x";
    for (uint256 i = 2 * length + 1; i > 1; --i) {
        buffer[i] = bytes1(uint8(48 + uint8(value & 15) + (uint8(value & 15) > 9 ? 39 : 0)));
        value >>= 4;
    }
    return string(buffer);
}
}

```

2. MongoDB Mongoose Schemas (Node.js)

These schemas map your original SQL database to MongoDB while integrating the new blockchain fields into the `Certificate` collection.

```

const mongoose = require('mongoose');

// =====
// 1. Course Schema
// =====
const courseSchema = new mongoose.Schema({
    courseid: { type: String, required: true, unique: true },
    coursename: { type: String, required: true },
    course_type: { type: String, required: true },
    course_abbr: { type: String, required: true },
    price: { type: Number, required: true, default: 0.00 },
    duration: { type: Number, required: true } // Duration in hours
});

const Course = mongoose.model('Course', courseSchema);

// =====
// 2. Student Schema
// =====
const studentSchema = new mongoose.Schema({
    sid: { type: String, required: true, unique: true },
    courseid: { type: String, ref: 'Course' },
    fname: { type: String, required: true },
    lname: { type: String, required: true },
    username: { type: String, required: true, unique: true },
    password: { type: String, required: true },
    passKey: { type: String, required: true },
    flag: { type: Number, required: true, default: 1 },

```

```

    email: { type: String, required: true, unique: true },
    phone: { type: String },
    city: { type: String },
    dob: { type: Date },
    photo: { type: String }
  }, { timestamps: { createdAt: 'created_at', updatedAt: false } });

const Student = mongoose.model('Student', studentSchema);

// =====
// 3. Certificate Schema (With Blockchain Data)
// =====
const certificateSchema = new mongoose.Schema({
  crtId: { type: String, required: true, unique: true },
  sid: { type: String, ref: 'Student', required: true },
  fullname: { type: String, required: true },
  email: { type: String, required: true },
  phone: { type: String, required: true },
  examId: { type: String },
  courseName: { type: String, required: true },
  score: { type: String, required: true },
  grade: { type: String, required: true },

  // Blockchain Specific Fields
  previous_hash: { type: String, required: true },
  current_hash: { type: String, required: true },
  transaction_details: { type: String, required: true },
  contract_address: { type: String, required: true }, // Where the contract is deployed
}, { timestamps: { createdAt: 'crt_generate_on', updatedAt: 'crt_update_on' } });

const Certificate = mongoose.model('Certificate', certificateSchema);

module.exports = { Course, Student, Certificate };

```

User prompt: Write the Express.js API endpoints that will connect this Mongoose database to the React frontend.

Response: Here is the complete Express.js backend implementation.

This setup integrates Mongoose for database interactions and `ethers.js` to communicate directly with your local Ethereum Ganache node, keeping the focus strictly on Ethereum's architecture rather than broader web networks.

Express.js & Ethereum API Server

You will need to install the required dependencies first. Run `npm install express mongoose cors ethers` in your project folder.

```

// server.js - Backend API Architecture by Saikat Sengupta
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const { ethers } = require('ethers');

// Import the Mongoose schemas created in the previous step
const { Course, Student, Certificate } = require('./models');

const app = express();

// Middleware to parse JSON payloads and enable cross-origin requests from React
app.use(cors());
app.use(express.json());

// =====
// Ethereum Setup (Ganache Node)
// =====

// Connect directly to the local Ganache RPC server
const provider = new ethers.JsonRpcProvider('http://127.0.0.1:7545');

// The private key of the admin wallet authorized to mint certificates (from Ganache)
const adminPrivateKey = 'YOUR_GANACHE_PRIVATE_KEY';
const wallet = new ethers.Wallet(adminPrivateKey, provider);

// The address where your Solidity contract was deployed
const contractAddress = 'YOUR_DEPLOYED_CONTRACT_ADDRESS';
const contractABI = [ /* Paste your compiled Solidity contract ABI array here */ ];

const certificateContract = new ethers.Contract(contractAddress, contractABI, wallet);

```

```

// =====
// REST API Endpoints
// =====

// 1. Fetch available courses for the React admission form
app.get('/api/courses', async (req, res) => {
  try {
    const courses = await Course.find();
    res.status(200).json(courses);
  } catch (error) {
    res.status(500).json({ error: 'Failed to fetch courses' });
  }
});

// 2. Process Student Admission
app.post('/api/students/admissions', async (req, res) => {
  try {
    const { sid, courseid, fname, lname, username, password, passKey, email, phone, city, dob } = req.body;

    // Save the student details directly to MongoDB
    const newStudent = new Student({
      sid, courseid, fname, lname, username, password, passKey, email, phone, city, dob
    });

    await newStudent.save();
    res.status(201).json({ message: 'Admission successful', student: newStudent });
  } catch (error) {
    res.status(400).json({ error: 'Admission failed', details: error.message });
  }
});

// 3. Mint Certificate on Ethereum and Sync to MongoDB
app.post('/api/certificates/mint', async (req, res) => {
  try {
    const { crtId, sid, fullname, email, phone, examid, coursename, score, grade } = req.body;
    const transactionDetails = `Minting certificate ${crtId} for ${fullname} (${coursename})`;

    // Step A: Send transaction to the Ethereum smart contract
    const tx = await certificateContract.mintCertificate(
      transactionDetails, sid, fullname, crtId, score, grade, coursename, phone, email
    );

    // Step B: Wait for the block to be confirmed on the Ganache network
    const receipt = await tx.wait();

    // Step C: Save the cryptographic proof alongside the student data in MongoDB
    const newCertificate = new Certificate({
      crtId, sid, fullname, email, phone, examid, coursename, score, grade,
      previous_hash: 'Handled internally by Solidity state',
      current_hash: receipt.blockHash, // Fetched directly from the Ethereum block
      transaction_details: receipt.hash, // The transaction ID
      contract_address: contractAddress
    });

    await newCertificate.save();

    res.status(201).json({
      message: 'Certificate permanently minted on Ethereum and synced to database.',
      transactionHash: receipt.hash,
      blockHash: receipt.blockHash
    });

  } catch (error) {
    res.status(500).json({ error: 'Ethereum minting failed', details: error.message });
  }
});

// 4. Verify Certificate Details
app.get('/api/certificates/verify/:crtid', async (req, res) => {
  try {
    // Fetch the certificate and populate linked student data
    const certificate = await Certificate.findOne({ crtId: req.params.crtid }).populate('sid', 'fname lname email');

    if (!certificate) {
      return res.status(404).json({ message: 'Certificate not found in database.' });
    }

    res.status(200).json(certificate);
  }
});

```

```

    } catch (error) {
      res.status(500).json({ error: 'Verification fetch failed', details: error.message });
    }
  });

// =====
// Database Connection & Server Initialization
// =====
const PORT = process.env.PORT || 5000;

mongoose.connect('mongodb://127.0.0.1:27017/studentPortal')
  .then(() => {
    console.log('Connected to MongoDB Student Portal Database.');
```

User prompt: How do I compile and deploy this Ethereum smart contract to my local Ganache network using Hardhat?

Response: s

Let's get this Ethereum contract deployed using Hardhat. We will configure Hardhat to connect specifically to your local Ganache network (usually running on port 7545) instead of its default internal network.

Here is the step-by-step procedure to compile and deploy your `StudentCertificatePortal` smart contract.

1 Project Setup

Initialize Hardhat in your project

Open your terminal in your project's root folder and install Hardhat along with the Hardhat Toolbox (which includes ethers v6).

```
npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox
npx hardhat init
```

When prompted, select **"Create a TypeScript project"** (or JavaScript, depending on your preference), and press enter to accept the defaults. This will create a `contracts/`, `scripts/`, and `test/` directory.

2 Configure Hardhat

Connect to your local Ganache node

Open the generated `hardhat.config.js` (or `.ts`) file in the root of your project. We need to add your Ganache network details.

Open the Ganache UI, click the key icon next to the first account, and copy the private key.

Replace the contents of `hardhat.config.js` with:

```
require("@nomicfoundation/hardhat-toolbox");

/** @type import('hardhat/config').HardhatUserConfig */
module.exports = {
  solidity: "0.8.19",
  networks: {
    ganache: {
      url: "http://127.0.0.1:7545", // Default Ganache GUI port
      accounts: [
        // Paste your Ganache private key here (ensure it starts with 0x)
        "0xYOUR_GANACHE_PRIVATE_KEY_HERE"
      ],
    },
  },
};
```

3 Add Your Smart Contract

Place the Solidity file

Navigate to the `contracts/` directory created by Hardhat. Delete the default `Lock.sol` file. Create a new file named `StudentCertificatePortal.sol` and paste the exact Solidity code from our previous step into it.

4 Compile the Contract

Compile Solidity to EVM Bytecode

Run the following command in your terminal to compile the smart contract:

```
npx hardhat compile
```

Note: This generates an `artifacts/` directory containing your contract's ABI (Application Binary Interface) which you will later import into your Express backend.

5 Create the Deployment Script

Write the deployment logic

Navigate to the `scripts/` directory, delete `deploy.js`, and create a new file named `deploy.js` (or `deploy.ts`). Add the following ethers.js code to handle the deployment:

```
const hre = require("hardhat");

async function main() {
  console.log("Starting deployment...");

  // Get the contract factory
  const CertificatePortal = await hre.ethers.getContractFactory("StudentCertificatePortal");

  // Deploy the contract
  const portal = await CertificatePortal.deploy();

  // Wait for the deployment transaction to be mined
  await portal.waitForDeployment();

  // Log the deployed address
  const targetAddress = await portal.getAddress();
  console.log(`StudentCertificatePortal successfully deployed to: ${targetAddress}`);
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

6 Execute Deployment

Deploy to your Ganache node

Ensure your Ganache UI application is open and running. Then, execute your deployment script, explicitly telling Hardhat to use the Ganache network you configured:

```
npx hardhat run scripts/deploy.js --network ganache
```

Your terminal will output something like: `StudentCertificatePortal successfully deployed to: 0x123abc...`

Crucial Next Step for your API: Take the deployed contract address from the terminal output and paste it into the `contractAddress` variable in your `server.js` Express file. Next, copy the ABI array found in `artifacts/contracts/StudentCertificatePortal.sol/StudentCertificatePortal.json` and paste it into the `contractABI` variable.

User prompt: Write the React components using Bootstrap to build the student admission and certificate verification forms.

Response: Here are the React components for the Student Admission and Certificate Verification forms. These use standard Bootstrap classes for a clean, responsive layout.

Prerequisites

Make sure you have Bootstrap installed in your React project. If you haven't already, run:

```
npm install bootstrap
```

Then, import it in your `src/index.js` or `src/App.js`:

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

1. Student Admission Form Component

This component captures the student data and submits it to the Express endpoint we created earlier. It includes a dynamic dropdown for courses.

```
import React, { useState, useEffect } from 'react';

const AdmissionForm = () => {
  const [courses, setCourses] = useState([]);
  const [formData, setFormData] = useState({
    sid: '', courseid: '', fname: '', lname: '', username: '',
    password: '', passKey: '', email: '', phone: '', city: '', dob: ''
  });
  const [status, setStatus] = useState({ type: '', message: '' });

  // Fetch available courses when component mounts
  useEffect(() => {
    fetch('http://localhost:5000/api/courses')
      .then(res => res.json())
      .then(data => setCourses(data))
  });
}
```

```

        .catch(err => console.error("Failed to fetch courses"));
    }, []);

    const handleChange = (e) => {
        setFormData({ ...formData, [e.target.name]: e.target.value });
    };

    const handleSubmit = async (e) => {
        e.preventDefault();
        try {
            const response = await fetch('http://localhost:5000/api/students/admissions', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify(formData)
            });
            const data = await response.json();

            if (response.ok) {
                setStatus({ type: 'success', message: 'Admission successful!' });
                // Reset form here if desired
            } else {
                setStatus({ type: 'danger', message: data.error || 'Admission failed.' });
            }
        } catch (error) {
            setStatus({ type: 'danger', message: 'Server error. Please try again later.' });
        }
    };

    return (
        <div className="container mt-5">
            <div className="card shadow">
                <div className="card-header bg-primary text-white">
                    <h3 className="mb-0">New Student Admission</h3>
                </div>
                <div className="card-body">
                    {status.message && (
                        <div className={`alert alert-${status.type}`} role="alert">
                            {status.message}
                        </div>
                    )}
                    <form onSubmit={handleSubmit}>
                        <div className="row mb-3">
                            <div className="col-md-6">
                                <label className="form-label">Student ID (SID)</label>
                                <input type="text" name="sid" className="form-control" onChange={handleChange} required />
                            </div>
                            <div className="col-md-6">
                                <label className="form-label">Select Course</label>
                                <select name="courseid" className="form-select" onChange={handleChange} required>
                                    <option value="">-- Choose a Course --</option>
                                    {courses.map(course => (
                                        <option key={course.courseid} value={course.courseid}>
                                            {course.coursename} ({course.course_type})
                                        </option>
                                    ))}
                                </select>
                            </div>
                        </div>

                        <div className="row mb-3">
                            <div className="col-md-6">
                                <label className="form-label">First Name</label>
                                <input type="text" name="fname" className="form-control" onChange={handleChange} required />
                            </div>
                            <div className="col-md-6">
                                <label className="form-label">Last Name</label>
                                <input type="text" name="lname" className="form-control" onChange={handleChange} required />
                            </div>
                        </div>

                        <div className="row mb-3">
                            <div className="col-md-4">
                                <label className="form-label">Username</label>
                                <input type="text" name="username" className="form-control" onChange={handleChange} required />
                            </div>
                            <div className="col-md-4">
                                <label className="form-label">Password</label>
                                <input type="password" name="password" className="form-control" onChange={handleChange} required />
                            </div>
                        </div>
                    </form>
                </div>
            </div>
        </div>
    );

```

```

        <div className="col-md-4">
          <label className="form-label">PassKey</label>
          <input type="text" name="passKey" className="form-control" onChange={handleChange} required />
        </div>
      </div>

      <div className="row mb-3">
        <div className="col-md-4">
          <label className="form-label">Email</label>
          <input type="email" name="email" className="form-control" onChange={handleChange} required />
        </div>
        <div className="col-md-4">
          <label className="form-label">Phone</label>
          <input type="text" name="phone" className="form-control" onChange={handleChange} />
        </div>
        <div className="col-md-4">
          <label className="form-label">Date of Birth</label>
          <input type="date" name="dob" className="form-control" onChange={handleChange} />
        </div>
      </div>

      <div className="mb-3">
        <label className="form-label">City</label>
        <input type="text" name="city" className="form-control" onChange={handleChange} />
      </div>

      <button type="submit" className="btn btn-success w-100">Submit Application</button>
    </form>
  </div>
  <div className="card-footer text-muted text-center small">
    Project Architecture by Saikat Sengupta
  </div>
</div>
</div>
);
};

export default AdmissionForm;

```

2. Certificate Verification Component

This component allows anyone to input a Certificate ID. It queries the Express API to fetch the MongoDB record, which includes the cryptographic proof generated by your Ethereum smart contract.

```

import React, { useState } from 'react';

const CertificateVerification = () => {
  const [crtid, setCrtid] = useState('');
  const [certData, setCertData] = useState(null);
  const [error, setError] = useState('');

  const handleVerify = async (e) => {
    e.preventDefault();
    setError('');
    setCertData(null);

    try {
      const response = await fetch(`http://localhost:5000/api/certificates/verify/${crtid}`);
      const data = await response.json();

      if (response.ok) {
        setCertData(data);
      } else {
        setError(data.message || 'Certificate not found.');
```

```

<form onSubmit={handleVerify} className="mb-4">
  <div className="input-group">
    <input
      type="text"
      className="form-control form-control-lg"
      placeholder="Enter Certificate ID (e.g., CRT-1001)"
      value={crtid}
      onChange={(e) => setCrtid(e.target.value)}
      required
    />
    <button className="btn btn-primary btn-lg" type="submit">Verify</button>
  </div>
</form>

{error && <div className="alert alert-danger">{error}</div>}

{certData && (
  <div className="alert alert-success border-success">
    <h5 className="alert-heading text-center mb-4">✅ Valid Certificate Found</h5>

    <div className="row mb-2">
      <div className="col-sm-4 fw-bold">Student Name:</div>
      <div className="col-sm-8">{certData.fullname}</div>
    </div>
    <div className="row mb-2">
      <div className="col-sm-4 fw-bold">Course:</div>
      <div className="col-sm-8">{certData.coursename}</div>
    </div>
    <div className="row mb-2">
      <div className="col-sm-4 fw-bold">Score / Grade:</div>
      <div className="col-sm-8">{certData.score} / {certData.grade}</div>
    </div>

    <hr />
    <h6 className="fw-bold mb-3">Cryptographic Proof (Ethereum)</h6>
    <div className="row mb-2">
      <div className="col-sm-4 fw-bold text-muted small">Block Hash:</div>
      <div className="col-sm-8 text-break font-monospace small">{certData.current_hash}</div>
    </div>
    <div className="row mb-2">
      <div className="col-sm-4 fw-bold text-muted small">Transaction:</div>
      <div className="col-sm-8 text-break font-monospace small">{certData.transaction_details}</div>
    </div>
    <div className="row mb-2">
      <div className="col-sm-4 fw-bold text-muted small">Contract:</div>
      <div className="col-sm-8 text-break font-monospace small">{certData.contract_address}</div>
    </div>
  </div>
)}
</div>
<div className="card-footer text-muted text-center small">
  Project Architecture by Saikat Sengupta
</div>
</div>
</div>
</div>
);
};

export default CertificateVerification;

```

User prompt: Write the React component for the Admin Panel that allows an administrator to input student grades and trigger the Web3 certificate minting process.

Response: Here is the React component for the Admin Minting Panel.

This interface allows an administrator to enter the finalized grades and trigger the Express API, which in turn signs the transaction and mints the certificate on the Ethereum blockchain.

Admin Panel Component (AdminMintingPanel.jsx)

```

import React, { useState } from 'react';

const AdminMintingPanel = () => {
  const [formData, setFormData] = useState({
    crtId: '', sid: '', fullname: '', email: '', phone: '',
    examid: '', coursename: '', score: '', grade: ''
  });

```

```

    });

    const [isMinting, setIsMinting] = useState(false);
    const [result, setResult] = useState(null);
    const [error, setError] = useState('');

    const handleChange = (e) => {
      setFormData({ ...formData, [e.target.name]: e.target.value });
    };

    const handleSubmit = async (e) => {
      e.preventDefault();
      setIsMinting(true);
      setError('');
      setResult(null);

      try {
        const response = await fetch('http://localhost:5000/api/certificates/mint', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify(formData)
        });

        const data = await response.json();

        if (response.ok) {
          setResult(data);
          // Clear form after successful minting
          setFormData({
            crtId: '', sid: '', fullname: '', email: '', phone: '',
            examId: '', coursename: '', score: '', grade: ''
          });
        } else {
          setError(data.error || 'Failed to mint certificate.');
```

```

</div>

<h5 className="border-bottom pb-2 mb-3 mt-4 text-primary">Student Identity</h5>
<div className="row mb-3">
  <div className="col-md-3">
    <label className="form-label fw-bold">Student ID (SID)</label>
    <input type="text" name="sid" className="form-control" value={formData.sid} onChange={handleChange}>
  </div>
  <div className="col-md-3">
    <label className="form-label fw-bold">Full Name</label>
    <input type="text" name="fullname" className="form-control" value={formData.fullname} onChange={handleChange}>
  </div>
  <div className="col-md-3">
    <label className="form-label fw-bold">Email</label>
    <input type="email" name="email" className="form-control" value={formData.email} onChange={handleChange}>
  </div>
  <div className="col-md-3">
    <label className="form-label fw-bold">Phone</label>
    <input type="text" name="phone" className="form-control" value={formData.phone} onChange={handleChange}>
  </div>
</div>

<h5 className="border-bottom pb-2 mb-3 mt-4 text-primary">Academic Results</h5>
<div className="row mb-4">
  <div className="col-md-6">
    <label className="form-label fw-bold">Total Score</label>
    <input type="text" name="score" className="form-control" placeholder="e.g., 850/1000" value={formData.score}>
  </div>
  <div className="col-md-6">
    <label className="form-label fw-bold">Final Grade</label>
    <select name="grade" className="form-select" value={formData.grade} onChange={handleChange} required>
      <option value="">-- Select Grade --</option>
      <option value="A+">A+</option>
      <option value="A">A</option>
      <option value="B+">B+</option>
      <option value="B">B</option>
      <option value="C">C</option>
      <option value="F">Fail</option>
    </select>
  </div>
</div>

<button
  type="submit"
  className="btn btn-primary btn-lg w-100 fw-bold shadow-sm"
  disabled={isMinting}>
  >
  {isMinting ? (
    <span className="spinner-border spinner-border-sm me-2" role="status" aria-hidden="true"></span>
    Minting on Ethereum Network...
  ) : (
    'Mint Certificate to Blockchain & Save to Database'
  )}
</button>
</form>
</div>
<div className="card-footer text-muted text-center small">
  Project Architecture by Saikat Sengupta
</div>
</div>
</div>
);
};

export default AdminMintingPanel;

```

User prompt: as needed

Response: Shifting the transaction signing to the frontend is a significant security upgrade. Instead of storing the admin's private key on the Node.js server, the admin will use the MetaMask browser extension to authorize and sign the minting transaction directly.

Here is how the architecture changes:

1 Frontend Prompts for Signature

MetaMask Connection

The React application uses `ethers.js` to connect to `window.ethereum` (MetaMask). When the admin clicks "Mint", MetaMask pops up asking them to sign the transaction and pay the gas fee.

2 Blockchain Confirmation

Wait for Ganache

React waits for the transaction to be mined on your local Ganache network and extracts the `transactionHash` and `blockHash` from the receipt.

3 Backend Database Sync

Save State

React sends the student data *along with the cryptographic proof* to a new Express.js endpoint, which simply saves it to MongoDB.

1. Update the React Frontend

First, install `ethers` in your React project if you haven't already:

```
npm install ethers
```

Update your `AdminMintingPanel.jsx` to handle the MetaMask connection and smart contract execution.

```
import React, { useState } from 'react';
import { ethers } from 'ethers';

// Replace with your actual deployed contract address and ABI
const CONTRACT_ADDRESS = 'YOUR_DEPLOYED_CONTRACT_ADDRESS';
const CONTRACT_ABI = [
  // Paste your ABI array here
  "function mintCertificate(string, string, string, string, string, string, string, string, string) public"
];

const AdminMintingPanel = () => {
  const [formData, setFormData] = useState({
    crtId: '', sid: '', fullname: '', email: '', phone: '',
    examId: '', courseName: '', score: '', grade: ''
  });

  const [walletAddress, setWalletAddress] = useState('');
  const [isMinting, setIsMinting] = useState(false);
  const [result, setResult] = useState(null);
  const [error, setError] = useState('');

  // Connect to MetaMask
  const connectWallet = async () => {
    if (window.ethereum) {
      try {
        const provider = new ethers.BrowserProvider(window.ethereum);
        const accounts = await provider.send("eth_requestAccounts", []);
        setWalletAddress(accounts[0]);
      } catch (err) {
        setError("User denied account access or MetaMask is locked.");
      }
    } else {
      setError("MetaMask is not installed. Please install it to continue.");
    }
  };

  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!walletAddress) return setError("Please connect your MetaMask wallet first.");

    setIsMinting(true);
    setError('');
    setResult(null);

    try {
      // 1. Initialize Ethers and Contract via MetaMask
      const provider = new ethers.BrowserProvider(window.ethereum);
      const signer = await provider.getSigner();
      const contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI, signer);

      const transactionDetails = `Minting certificate ${formData.crtId} for ${formData.fullname} (${formData.courseName}`

      // 2. Trigger the MetaMask popup for the admin to sign
```

```

const tx = await contract.mintCertificate(
  transactionDetails, formData.sid, formData.fullname, formData.crtid,
  formData.score, formData.grade, formData.coursename, formData.phone, formData.email
);

// 3. Wait for the block to be mined on Ganache
const receipt = await tx.wait();

// 4. Send the data AND the blockchain receipt to the Express backend for MongoDB storage
const dbResponse = await fetch('http://localhost:5000/api/certificates/save', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    ...formData,
    transactionHash: receipt.hash,
    blockHash: receipt.blockHash,
    contractAddress: CONTRACT_ADDRESS
  })
});

if (dbResponse.ok) {
  setResult({
    message: 'Certificate minted via MetaMask and saved to Database.',
    transactionHash: receipt.hash,
    blockHash: receipt.blockHash
  });
  setFormData({ crtid: '', sid: '', fullname: '', email: '', phone: '', examid: '', coursename: '', score: '' });
} else {
  setError('Minted on blockchain, but failed to save to database.');
```

```

} catch (err) {
  setError(`Transaction failed: ${err.message}`);
} finally {
  setIsMinting(false);
}
};

return (
  <div className="container mt-5 mb-5">
    <div className="card shadow border-dark">
      <div className="card-header bg-dark text-white d-flex justify-content-between align-items-center">
        <h4 className="mb-0">Admin Panel: MetaMask Minting</h4>
        {walletAddress ? (
          <span className="badge bg-success">Connected: {walletAddress.substring(0, 6)}...{walletAddress.substring(
            6, 10
          )}</span> : (
            <button className="btn btn-warning btn-sm fw-bold" onClick={connectWallet}>Connect MetaMask</button>
          )
        )}
      </div>

      <div className="card-body bg-light">
        {error && <div className="alert alert-danger">{error}</div>}
        {result && (
          <div className="alert alert-success shadow-sm">
            <h5 className="alert-heading">🎉 {result.message}</h5>
            <hr />
            <div className="small font-monospace">
              <strong>Transaction Hash:</strong> {result.transactionHash}<br />
              <strong>Block Hash:</strong> {result.blockHash}
            </div>
          </div>
        )}

        <form onSubmit={handleSubmit}>
          {/* Form inputs remain exactly the same as the previous version */}
          {/* ... (Certificate, Student Identity, Academic Results inputs) ... */}

          <button
            type="submit"
            className="btn btn-primary btn-lg w-100 fw-bold shadow-sm mt-4"
            disabled={isMinting || !walletAddress}
          >
            {isMinting ? (
              <>
                <span className="spinner-border spinner-border-sm me-2" role="status" aria-hidden="true"></span>
                Waiting for MetaMask Signature...
              </>
            ) : (
              'Sign & Mint via MetaMask'
            )}
          </button>
        </form>
      </div>
    </div>
  </div>
);

```

```

        </button>
      </form>
    </div>
    <div className="card-footer text-muted text-center small">
      Project Architecture by Saikat Sengupta
    </div>
  </div>
</div>
);
};

```

```
export default AdminMintingPanel;
```

2. Update the Express.js Backend

Your backend no longer needs to hold the `adminPrivateKey` or the `ethers.Wallet` logic. It simply needs a new endpoint to accept the blockchain metadata from the React frontend and save it to MongoDB.

Add this new endpoint to your `server.js` file:

```

// POST: Save certificate to MongoDB after it has been minted via MetaMask on the frontend
app.post('/api/certificates/save', async (req, res) => {
  try {
    const {
      crtid, sid, fullname, email, phone, examid, coursename, score, grade,
      transactionHash, blockHash, contractAddress
    } = req.body;

    // Save the cryptographic proof alongside the student data
    const newCertificate = new Certificate({
      crtid, sid, fullname, email, phone, examid, coursename, score, grade,
      previous_hash: 'Handled internally by Solidity state',
      current_hash: blockHash,
      transaction_details: transactionHash,
      contract_address: contractAddress
    });

    await newCertificate.save();

    res.status(201).json({ message: 'Certificate saved to database successfully.' });
  } catch (error) {
    res.status(500).json({ error: 'Database save failed', details: error.message });
  }
});

```

MetaMask Configuration Note: To test this locally, you must open MetaMask, go to Settings > Networks > Add Network, and add your Ganache network manually (RPC URL: `[http://127.0.0.1:7545](http://127.0.0.1:7545)`, Chain ID: 1337). You will also need to import one of the Ganache private keys into MetaMask so you have a wallet with test ETH to pay for the minting gas fees.

User prompt: now guide me step by step creation of folder and file management of above project creation

Response: This modular folder structure separates your Ethereum smart contracts, Express backend, and React frontend into distinct environments to prevent dependency conflicts.

Final Directory Map

```

student-education-portal/
├── ethereum/                                # Hardhat & Solidity
│   ├── contracts/
│   │   └── StudentCertificatePortal.sol
│   ├── scripts/
│   │   └── deploy.js
│   └── hardhat.config.js
├── backend/                                # Node.js, Express, MongoDB
│   ├── models.js                          # Mongoose schemas
│   ├── server.js                          # API endpoints
│   └── package.json
└── frontend/                              # React UI
    ├── src/
    │   ├── components/
    │   │   ├── AdmissionForm.jsx
    │   │   ├── AdminMintingPanel.jsx
    │   │   └── CertificateVerification.jsx
    │   ├── App.js
    │   └── index.js
    └── package.json

```

1 Root Folder Setup

Create the master project container

Open your terminal and run the following commands to create the main directory that will house all three parts of the architecture.

```
mkdir student-education-portal
cd student-education-portal
```

2 1. Ethereum Smart Contract Directory

Set up Hardhat environment

Create the Ethereum workspace, initialize it, and create the required files.

```
mkdir ethereum
cd ethereum
npm init -y
npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox
npx hardhat init
```

- Choose **Create an empty hardhat.config.js** (or TypeScript if preferred).
- Create the `contracts` folder and add `StudentCertificatePortal.sol`.
- Create the `scripts` folder and add `deploy.js`.
- Update `hardhat.config.js` with your Ganache network settings.
- Run `cd ..` to return to the root folder.

3 2. Backend Directory

Set up Express API and Mongoose models

Create the backend server environment and install dependencies.

```
mkdir backend
cd backend
npm init -y
npm install express mongoose cors
```

- Create `models.js` and paste the Mongoose schema code (Course, Student, Certificate).
- Create `server.js` and paste the Express API and MongoDB connection code.
- Run `cd ..` to return to the root folder.

4 3. Frontend Directory

Set up React and UI components

Scaffold the React application and install Bootstrap and Ethers.js.

```
npx create-react-app frontend
cd frontend
npm install bootstrap ethers
```

- Navigate into `src/`. Create a new folder named `components/`.
- Inside `components/`, create three files: `AdmissionForm.jsx`, `AdminMintingPanel.jsx`, and `CertificateVerification.jsx`.
- Open `src/index.js` and add `import 'bootstrap/dist/css/bootstrap.min.css';` at the top.
- Open `src/App.js`, remove the default boilerplate, and import/render your three components.

User prompt: react app will be generate using vite

Response: Here is the updated setup process using Vite. Vite is significantly faster than Create React App and uses slightly different file extensions (like `.jsx` instead of `.js` for components) and entry points.

Updated Frontend Directory Structure

```
frontend/                                # React UI (Vite)
├── src/
│   ├── components/
│   │   ├── AdmissionForm.jsx
│   │   ├── AdminMintingPanel.jsx
│   │   └── CertificateVerification.jsx
│   ├── App.jsx                          # Main app component
│   └── main.jsx                          # Vite's entry point (instead of index.js)
└── index.html
```

```
└─ vite.config.js
└─ package.json
```

Step 3: Frontend Directory (Vite Setup)

Run these commands from your master `student-education-portal` folder to scaffold the React application using Vite, install dependencies, and create the component structure.

```
# Scaffold the Vite project
npm create vite@latest frontend -- --template react

# Navigate in and install default dependencies
cd frontend
npm install

# Install the required packages for this architecture
npm install bootstrap ethers
```

Once the installations are complete, set up your files:

- 1. Create Component Files:** Inside `src/`, create a `components/` folder and add your three files: `AdmissionForm.jsx`, `AdminMintingPanel.jsx`, and `CertificateVerification.jsx`.
- 2. Import Bootstrap:** Open `src/main.jsx` (Vite's equivalent of `index.js`) and add the Bootstrap CSS import at the very top:

```
import 'bootstrap/dist/css/bootstrap.min.css';
```
- 3. Clean up the App File:** Open `src/App.jsx`, delete the default Vite boilerplate, and import your new components to render them.

User prompt: What are the exact terminal commands I need to run simultaneously to start the Ganache node, Express backend, and Vite frontend for testing?

Response: To run this full-stack Ethereum architecture locally, you will need to open **three separate terminal windows** (or tabs) from your master `student-education-portal` folder.

Before starting, ensure your **MongoDB server is running** locally on its default port (27017), and your **Ganache UI application is open** and running on port 7545.

Terminal 1: Smart Contract Deployment (Ethereum)

You only need to run this once whenever you restart Ganache or change your Solidity code.

```
cd ethereum

# Compile the smart contract
npx hardhat compile

# Deploy to your local Ganache network
npx hardhat run scripts/deploy.js --network ganache
```

(Remember to copy the deployed contract address from the output and update your frontend `AdminMintingPanel.jsx` and backend `server.js` before proceeding).

Terminal 2: The Backend (Node.js & Express)

This terminal will stay open to listen for API requests and connect to MongoDB.

```
cd backend

# Start the Express server
node server.js
```

Expected output: "Connected to MongoDB Student Portal Database. Server running on port 5000"

Terminal 3: The Frontend (React & Vite)

This terminal will stay open to run the Vite development server for your user interface.

```
cd frontend

# Start the Vite development server
npm run dev
```

Expected output: A local host link (usually `http://localhost:5173`). Click it or paste it into your browser to view the application.

User prompt: during create react apps using vite showing two option Oxlint ESLint which option will choose for this project

Response: For this project, you should choose **ESLint**.

Here is why it is the best fit for your architecture:

- **React Hooks Enforcement:** You are using `useState` and `useEffect` extensively in your components (especially for fetching courses and handling MetaMask state). ESLint comes natively configured to warn you if you miss dependencies in your hooks, preventing hard-to-find bugs.
- **Industry Standard:** It has a massive ecosystem. If you want to add specific linting rules later (like formatting rules for Web3/Ethers.js), ESLint has plugins for almost everything.
- **Community Support:** If you run into an error message, pasting an ESLint error into Google will instantly yield thousands of solutions.

Oxlint is a fantastic, blazing-fast new linter written in Rust, but it is still relatively new. It focuses on parsing speed over a massive plugin ecosystem. For a robust React and Web3 project, the mature ecosystem of ESLint is much safer.

Just use your arrow keys to highlight **ESLint** and press Enter to continue your Vite setup!

User prompt: G:\mongoDB\student-education-portal\ethereum>npx hardhat init Error HHE3: No Hardhat config file found. You can initialize a new project by running Hardhat with --init For more info go to <https://hardhat.org/HHE3> or run Hardhat with --show-stack-traces G:\mongoDB\student-education-portal\ethereum>

Response: This happens when `npx` picks up a version of Hardhat that expects a slightly different initialization command.

To trigger the setup wizard, run it without the `init` keyword, or use the flag the error message suggested.

Try this first:

```
npx hardhat
```

If that still gives you an error, run exactly what the error suggested:

```
npx hardhat --init
```

Once the interactive menu appears:

1. Select **"Create a JavaScript project"** (or TypeScript if you prefer).
2. Press **Enter** to accept the default project root.
3. Press **Enter** to add a `.gitignore`.
4. Press **Enter** to install the sample project's dependencies if prompted.

After it finishes, you will see the `contracts/`, `scripts/`, and `hardhat.config.js` files generated in your folder.

User prompt: adk Please provide either a relative or an absolute path: » after runing npx hardhat --init

Response: Hardhat is just asking you exactly where it should save all the new project files.

Since you are already inside the `ethereum` folder where you want the project to live, you can just tell it to use the "current directory".

Type a single dot:

```
.
```

(or type `./`)

Then press **Enter**.

This tells Hardhat to initialize everything right inside `G:\mongoDB\student-education-portal\ethereum`.

If it prompts you with other questions like "Do you want to add a `.gitignore`?" or "Do you want to install this sample project's dependencies?", just press **Enter** to say Yes to all of them.

User prompt: A TypeScript Hardhat project using Node Test Runner and Viem A TypeScript Hardhat project using Mocha and Ethers.js A minimal Hardhat project

Response: You should choose: **A TypeScript Hardhat project using Mocha and Ethers.js** (or if you see a "JavaScript project" option higher up in the menu, pick that!).

Here is why: Throughout this project, we built your Node.js backend and your React frontend using **Ethers.js** to communicate with the Ethereum blockchain and MetaMask. Choosing this option ensures Hardhat installs the correct `ethers` plugins to match the rest of your architecture. (Viem is a different library and would conflict with our code).

What to do next:

1. Use your arrow keys to highlight the **Mocha and Ethers.js** option (or JavaScript) and press **Enter**.
2. For all the following questions (like "Do you want to add a `.gitignore`?" or "Do you want to install this sample project's dependencies?"), just press **Enter** to accept the defaults.

Note: If Hardhat creates files ending in .ts (like `hardhat.config.ts` or `deploy.ts`), don't worry! You can either rename them to .js, or just paste the JavaScript code I provided earlier directly into them—it will work perfectly.

User prompt: Hardhat only supports ESM projects. Would you like to change ".\package.json" to turn your project into ESM? (Y/n) » true

Response: Yes, pressing **Enter** with true (or typing Y) is perfectly fine! It updates your `package.json` to use "type": "module", which is the modern JavaScript standard.

However, because your project is now an ESM (ECMAScript Module) project, the older `require()` and `module.exports` syntax I provided earlier will cause an error.

To fix this, you just need to use `import` and `export default` instead. Here is the updated code for your two Hardhat files.

1. Updated `hardhat.config.ts` (or .js)

Replace the contents of your config file with this ESM-compatible version:

```
import "@nomicfoundation/hardhat-toolbox";

/** @type import('hardhat/config').HardhatUserConfig */
const config = {
  solidity: "0.8.19",
  networks: {
    ganache: {
      url: "http://127.0.0.1:7545",
      accounts: [
        // Paste your Ganache private key here (ensure it starts with 0x)
        "0xYOUR_GANACHE_PRIVATE_KEY_HERE"
      ],
    },
  },
};

export default config;
```

2. Updated `scripts/deploy.ts` (or .js)

Replace the contents of your deployment script with this ESM-compatible version:

```
import hre from "hardhat";

async function main() {
  console.log("Starting deployment...");

  // Get the contract factory
  const CertificatePortal = await hre.ethers.getContractFactory("StudentCertificatePortal");

  // Deploy the contract
  const portal = await CertificatePortal.deploy();

  // Wait for the deployment transaction to be mined
  await portal.waitForDeployment();

  // Log the deployed address
  const targetAddress = await portal.getAddress();
  console.log(`StudentCertificatePortal successfully deployed to: ${targetAddress}`);
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

Once you have saved these updated files, you can proceed with the compile and deploy commands from earlier:

```
npx hardhat compile
npx hardhat run scripts/deploy.ts --network ganache
```

User prompt: npm error code ENOENT npm error syscall spawn git npm error path git npm error errno -4058 npm error enoent An unknown git error occurred npm error enoent This is related to npm not being able to find a file. npm error enoent npm error A complete log of this run can be found in: C:\Users\nieli\AppData\Local\npm-cache_logs\2026-09-08T09_52_33_327Z-debug-0.log Error HHE1400: Failed to install the project dependencies. Please try again. If it persists, please open an issue in the Hardhat repository. For more info go to <https://hardhat.org/HHE1400> or run Hardhat with --show-stack-traces

Response: This error happens because **npm is trying to use Git** to download some background dependencies for Hardhat, but it cannot find Git installed on your Windows computer (or it isn't registered in your system's PATH).

Here is how to fix this:

1 Download and Install Git

Go to the official Git website (<https://git-scm.com/download/win>) and download the "64-bit Git for Windows Setup".

Run the installer. You can just click **"Next"** through all the prompts and leave everything on the default settings (the default settings automatically add Git to your system PATH, which is what npm needs).

2 Restart Your Terminal

Crucial step - don't skip

Your current terminal window will not recognize that Git has been installed until it restarts.

Close your command prompt (or if you are using VS Code, close the integrated terminal by clicking the trash can icon). Open a brand new terminal window and navigate back to your `ethereum` folder:

```
cd G:\mongoDB\student-education-portal\ethereum
```

3 Install Dependencies

Since the Hardhat setup failed during the installation step, you just need to tell npm to try installing the dependencies again manually:

```
npm install
```

Once that finishes without errors, your Hardhat environment is fully ready. You can then proceed with creating your `hardhat.config.js` and compiling your smart contract!

User prompt: G:\mongoDB\student-education-portal\ethereum>npx hardhat init Hardhat only supports ESM projects. Please make sure you have "type": "module" in your package.json. You can set it automatically by running: npm pkg set type="module"

Response: This is a quick and easy fix! Hardhat is just enforcing the modern ESM standard we discussed a moment ago, but it needs your `package.json` file to explicitly state it before it will let you initialize the project.

Here is how to resolve it:

1 Set the Module Type

Update your package file

Run the exact command the error message suggested to update your `package.json` file:

```
npm pkg set type="module"
```

2 Re-run Hardhat Init

Trigger the setup wizard

Now that the package file is updated, you can start the Hardhat setup process again:

```
npx hardhat init
```

3 Complete the Setup

Follow the prompts

When the interactive menu appears, follow the same steps as before:

1. Select **"A TypeScript project using Mocha and Ethers.js"** (or JavaScript if it appears).
2. Type `.` and press Enter for the project root.
3. Press Enter to accept the `.gitignore` and dependency installations.

Once it finishes successfully, you can copy in the `StudentCertificatePortal.sol` file and use the updated ESM scripts I provided earlier!

User prompt: ReferenceError: require is not defined in ES module scope, you can use import instead This file is being treated as an ES module because it has a '.js' file extension and '\\?\\G:\\mongoDB\\student-education-portal\\ethereum\\package.json' contains "type": "module". To treat it as a CommonJS script, rename it to use the '.cjs' file extension.

Response: This error happens because Hardhat generated the default files using the older `require()` syntax, but your project is now configured to use the modern ES Module standard (`"type": "module"`).

To fix this, you just need to replace the `require()` statements with `import` statements in two specific files.

1. Update `hardhat.config.js`

Open your `hardhat.config.js` file and replace everything inside it with this updated ESM code:

```
import "@nomicfoundation/hardhat-toolbox";

/** @type import('hardhat/config').HardhatUserConfig */
const config = {
  solidity: "0.8.19", // Make sure this matches your contract's version
  networks: {
    ganache: {
      url: "http://127.0.0.1:7545",
      accounts: [
        // Paste your Ganache private key here (ensure it starts with 0x)
        "0xYOUR_GANACHE_PRIVATE_KEY_HERE"
      ],
    },
  },
};

export default config;
```

2. Update scripts/deploy.js

Open your `scripts/deploy.js` file and replace everything inside it with this ESM code:

```
import hre from "hardhat";

async function main() {
  console.log("Starting deployment...");

  // Get the contract factory
  const CertificatePortal = await hre.ethers.getContractFactory("StudentCertificatePortal");

  // Deploy the contract
  const portal = await CertificatePortal.deploy();

  // Wait for the deployment transaction to be mined
  await portal.waitForDeployment();

  // Log the deployed address
  const targetAddress = await portal.getAddress();
  console.log(`StudentCertificatePortal successfully deployed to: ${targetAddress}`);
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

Once you have saved these changes, your deployment commands will work perfectly:

```
npx hardhat compile
npx hardhat run scripts/deploy.js --network ganache
```

User prompt: generate the app.js content for react using react router dom navigation

Response: To set up navigation between your components, we will use `react-router-dom` to create a single-page application (SPA) with a Bootstrap navigation bar.

Here is how to set it up step by step.

1 Install React Router

Required for routing

Open your terminal, ensure you are inside the `frontend` directory, and install the package:

```
npm install react-router-dom
```

2 Update the App Component

`src/App.jsx`

Open your `src/App.jsx` file (or `src/App.js`), delete the existing boilerplate, and paste the following code. This creates a top navigation bar and maps URLs to your three specific components.

App.jsx Code

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route, NavLink } from 'react-router-dom';

// Import the components we created earlier
import AdmissionForm from '../components/AdmissionForm';
```

```
import CertificateVerification from './components/CertificateVerification';
import AdminMintingPanel from './components/AdminMintingPanel';

const App = () => {
  return (
    <Router>
      <div className="min-vh-100 bg-light pb-5">

        { /* Main Navigation Bar */ }
        <nav className="navbar navbar-expand-lg navbar-dark bg-dark mb-4 shadow-sm">
          <div className="container">
            <span className="navbar-brand fw-bold">EduPortal Web3</span>

            { /* Mobile toggle button */ }
            <button
              className="navbar-toggler"
              type="button"
              data-bs-toggle="collapse"
              data-bs-target="#navbarNav"
            >
              <span className="navbar-toggler-icon"></span>
            </button>

            <div className="collapse navbar-collapse" id="navbarNav">
              <ul className="navbar-nav me-auto">
                <li className="nav-item">
                  <NavLink
                    to="/"
                    className={({ isActive }) => isActive ? "nav-link active fw-bold" : "nav-link"}
                  >
                    Student Admission
                  </NavLink>
                </li>
                <li className="nav-item">
                  <NavLink
                    to="/verify"
                    className={({ isActive }) => isActive ? "nav-link active fw-bold" : "nav-link"}
                  >
                    Verify Certificate
                  </NavLink>
                </li>
              </ul>

              { /* Admin Portal Link aligned to the right */ }
              <ul className="navbar-nav">
                <li className="nav-item">
                  <NavLink
                    to="/admin"
                    className={({ isActive }) => isActive ? "nav-link active text-warning fw-bold" : "nav-link text-
                    <img alt="gear icon" data-bbox="178 645 192 655"/> Admin Portal
                  </NavLink>
                </li>
              </ul>
            </div>
          </div>
        </nav>

        { /* Dynamic Page Content Routing */ }
        <div className="container">
          <Routes>
            <Route path="/" element={ <AdmissionForm /> } />
            <Route path="/verify" element={ <CertificateVerification /> } />
            <Route path="/admin" element={ <AdminMintingPanel /> } />
          </Routes>
        </div>

        { /* Global Footer */ }
        <footer className="text-center mt-5 pt-4 border-top text-muted small">
          <p>Project Architecture by Saikat Sengupta</p>
        </footer>

      </div>
    </Router>
  );
};

export default App;
```

How this works:

1. **BrowserRouter (Router)**: Wraps the entire application to enable URL tracking without refreshing the page.
 2. **NavLink**: Used instead of standard HTML `<a href="...">` tags. It automatically detects if the user is on that specific route and allows us to apply an `active` CSS class to highlight the current page in the navbar.
 3. **Routes & Route**: Acts as a switchboard. When the URL changes to `/admin`, it unmounts the admission form and mounts the `AdminMintingPanel` component dynamically.
-

User prompt: used proper bootstrap css for each pages and main page app.js

Response: To give your application a polished, professional, and cohesive look, we will deeply integrate modern Bootstrap 5 utility classes (shadows, spacing, responsive grids, and input groups) across all your components.

I have also added the **Bootstrap JS bundle** to your `App.jsx` so that the mobile navigation hamburger menu functions correctly.

Here is the complete, styled code for all four core files.

1. The Main Router (`src/App.jsx`)

This updated version includes a sticky navbar, improved active state styling, and the Bootstrap JavaScript required for the mobile dropdown menu.

```
import React, { useEffect } from 'react';
import { BrowserRouter as Router, Routes, Route, NavLink } from 'react-router-dom';

// Import Bootstrap JS for the mobile navbar toggle
import 'bootstrap/dist/js/bootstrap.bundle.min.js';

import AdmissionForm from './components/AdmissionForm';
import CertificateVerification from './components/CertificateVerification';
import AdminMintingPanel from './components/AdminMintingPanel';

const App = () => {
  return (
    <Router>
      <div className="d-flex flex-column min-vh-100 bg-light">

        {/* Sticky Header Navigation */}
        <nav className="navbar navbar-expand-lg navbar-dark bg-dark sticky-top shadow-sm py-3">
          <div className="container">
            <span className="navbar-brand fw-bold fs-4 text-primary">
              Edu<span className="text-white">Portal</span> Web3
            </span>

            <button
              className="navbar-toggler"
              type="button"
              data-bs-toggle="collapse"
              data-bs-target="#mainNavbar"
            >
              <span className="navbar-toggler-icon"></span>
            </button>

            <div className="collapse navbar-collapse" id="mainNavbar">
              <ul className="navbar-nav me-auto mb-2 mb-lg-0 ms-4">
                <li className="nav-item me-2">
                  <NavLink to="/" className={({ isActive }) => isActive ? "nav-link active fw-bold border-bottom bor"
                    Student Admission
                  </NavLink>
                </li>
                <li className="nav-item">
                  <NavLink to="/verify" className={({ isActive }) => isActive ? "nav-link active fw-bold border-bott"
                    Verify Certificate
                  </NavLink>
                </li>
              </ul>

              <ul className="navbar-nav">
                <li className="nav-item bg-black rounded shadow-sm">
                  <NavLink to="/admin" className={({ isActive }) => isActive ? "nav-link active text-warning fw-bold"
                    <img alt="gear icon" data-bbox="221 878 235 892"/> Admin Minting
                  </NavLink>
                </li>
              </ul>
            </div>
          </div>
        </nav>

        <Routes>
          <Route path="/" element={AdmissionForm} />
          <Route path="/verify" element={CertificateVerification} />
          <Route path="/admin" element={AdminMintingPanel} />
        </Routes>
      </div>
    </Router>
  );
};
```

```

    </nav>

    { /* Main Content Area */ }
    <main className="container flex-grow-1 py-5">
      <Routes>
        <Route path="/" element={ <AdmissionForm /> } />
        <Route path="/verify" element={ <CertificateVerification /> } />
        <Route path="/admin" element={ <AdminMintingPanel /> } />
      </Routes>
    </main>

    { /* Footer */ }
    <footer className="bg-white py-4 mt-auto border-top text-center text-muted shadow-sm">
      <div className="container">
        <small className="fw-semibold">Document design by Saikat Sengupta</small>
      </div>
    </footer>

  </div>
</Router>
);
};

export default App;

```

2. The Admission Form (src/components/AdmissionForm.jsx)

This features a clean card layout with an organized grid system, making the long form much easier to read and fill out.

```

import React, { useState, useEffect } from 'react';

const AdmissionForm = () => {
  const [courses, setCourses] = useState([]);
  const [formData, setFormData] = useState({
    sid: '', courseid: '', fname: '', lname: '', username: '',
    password: '', passKey: '', email: '', phone: '', city: '', dob: ''
  });
  const [status, setStatus] = useState({ type: '', message: '' });

  useEffect(() => {
    fetch('http://localhost:5000/api/courses')
      .then(res => res.json())
      .then(data => setCourses(data))
      .catch(err => console.error("Failed to fetch courses"));
  }, []);

  const handleChange = (e) => setFormData({ ...formData, [e.target.name]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      const response = await fetch('http://localhost:5000/api/students/admissions', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(formData)
      });
      const data = await response.json();

      if (response.ok) {
        setStatus({ type: 'success', message: '🎉 Admission application submitted successfully!' });
      } else {
        setStatus({ type: 'danger', message: data.error || 'Admission failed.' });
      }
    } catch (error) {
      setStatus({ type: 'danger', message: 'Server connection error. Please try again later.' });
    }
  };

  return (
    <div className="row justify-content-center">
      <div className="col-lg-10">
        <div className="card shadow-lg border-0 rounded-4">
          <div className="card-header bg-primary text-white p-4 border-0 rounded-top-4">
            <h3 className="mb-0 fw-bold">🎓 New Student Admission</h3>
            <p className="mb-0 text-white-50 small">Fill out the details below to register for a new course.</p>
          </div>

          <div className="card-body p-5">
            {status.message && (

```

```

<div className={`alert alert-${status.type} alert-dismissible fade show shadow-sm`} role="alert">
  {status.message}
  <button type="button" className="btn-close" onClick={() => setStatus({type:'', message:''})}></button>
</div>
)}

<form onSubmit={handleSubmit}>
  <h5 className="text-secondary mb-3 border-bottom pb-2">Academic Selection</h5>
  <div className="row g-3 mb-4">
    <div className="col-md-6">
      <label className="form-label fw-semibold text-muted">Student ID (SID)</label>
      <input type="text" name="sid" className="form-control form-control-lg bg-light" placeholder="e.g.,
    </div>
    <div className="col-md-6">
      <label className="form-label fw-semibold text-muted">Select Course</label>
      <select name="courseid" className="form-select form-select-lg bg-light" onChange={handleChange} re
        <option value="">-- Choose a Course --</option>
        {courses.map(course => (
          <option key={course.courseid} value={course.courseid}>
            {course.coursename} ({course.course_type})
          </option>
        ))}
      </select>
    </div>
  </div>

  <h5 className="text-secondary mb-3 border-bottom pb-2 mt-5">Personal Information</h5>
  <div className="row g-3 mb-4">
    <div className="col-md-6">
      <label className="form-label fw-semibold text-muted">First Name</label>
      <input type="text" name="fname" className="form-control" onChange={handleChange} required />
    </div>
    <div className="col-md-6">
      <label className="form-label fw-semibold text-muted">Last Name</label>
      <input type="text" name="lname" className="form-control" onChange={handleChange} required />
    </div>
    <div className="col-md-4">
      <label className="form-label fw-semibold text-muted">Date of Birth</label>
      <input type="date" name="dob" className="form-control" onChange={handleChange} />
    </div>
    <div className="col-md-4">
      <label className="form-label fw-semibold text-muted">Phone Number</label>
      <input type="text" name="phone" className="form-control" onChange={handleChange} />
    </div>
    <div className="col-md-4">
      <label className="form-label fw-semibold text-muted">City</label>
      <input type="text" name="city" className="form-control" onChange={handleChange} />
    </div>
  </div>

  <h5 className="text-secondary mb-3 border-bottom pb-2 mt-5">Account Credentials</h5>
  <div className="row g-3 mb-5">
    <div className="col-md-6">
      <label className="form-label fw-semibold text-muted">Email Address</label>
      <input type="email" name="email" className="form-control" placeholder="student@example.com" onChan
    </div>
    <div className="col-md-6">
      <label className="form-label fw-semibold text-muted">Username</label>
      <input type="text" name="username" className="form-control" onChange={handleChange} required />
    </div>
    <div className="col-md-6">
      <label className="form-label fw-semibold text-muted">Password</label>
      <input type="password" name="password" className="form-control" onChange={handleChange} required />
    </div>
    <div className="col-md-6">
      <label className="form-label fw-semibold text-muted">Recovery PassKey</label>
      <input type="text" name="passKey" className="form-control" onChange={handleChange} required />
    </div>
  </div>

  <div className="text-end mt-4">
    <button type="submit" className="btn btn-primary btn-lg px-5 shadow-sm fw-bold rounded-pill">
      Submit Application
    </button>
  </div>
</form>
</div>
</div>
</div>

```

```

    </div>
  );
};

```

```
export default AdmissionForm;
```

3. The Certificate Verification (src/components/CertificateVerification.jsx)

This component uses Bootstrap's input-group for a prominent search bar and renders the results in a stylized layout that mimics a verifiable document.

```

import React, { useState } from 'react';

const CertificateVerification = () => {
  const [crtid, setCrtid] = useState('');
  const [certData, setCertData] = useState(null);
  const [error, setError] = useState('');

  const handleVerify = async (e) => {
    e.preventDefault();
    setError('');
    setCertData(null);

    try {
      const response = await fetch(`http://localhost:5000/api/certificates/verify/${crtid}`);
      const data = await response.json();
      if (response.ok) {
        setCertData(data);
      } else {
        setError(data.message || 'Certificate not found.');
```

```

      } catch (err) {
        setError('Server error while verifying certificate.');
```

```

    }
  };

```

```

  return (
    <div className="row justify-content-center mt-4">
      <div className="col-lg-8">

        <div className="text-center mb-5">
          <h2 className="fw-bold">Web3 Certificate Verification</h2>
          <p className="text-muted">Enter a certificate ID to instantly verify its authenticity on the Ethereum bloc</p>
        </div>

        <form onSubmit={handleVerify} className="mb-5 shadow-sm rounded-pill p-2 bg-white">
          <div className="input-group input-group-lg">
            <input
              type="text"
              className="form-control border-0 bg-transparent px-4"
              placeholder="Enter Certificate ID (e.g., CRT-1001)"
              value={crtid}
              onChange={(e) => setCrtid(e.target.value)}
              required
            />
            <button className="btn btn-success rounded-pill px-5 fw-bold shadow-sm" type="submit">
              Verify Now
            </button>
          </div>
        </form>

        {error && (
          <div className="alert alert-danger shadow-sm text-center fw-semibold rounded-4 py-3 border-0">
            ✖ {error}
          </div>
        )}

        {certData && (
          <div className="card shadow-lg border-0 rounded-4 overflow-hidden mt-4">
            <div className="bg-success bg-opacity-10 text-success p-4 border-bottom border-success border-opacity-25">
              <h4 className="mb-0 fw-bold">✔ Verified Authentic</h4>
              <span className="badge bg-success py-2 px-3 fs-6 rounded-pill">Blockchain Secured</span>
            </div>

            <div className="card-body p-5">
              <div className="row mb-4">
                <div className="col-sm-4 text-muted fw-semibold mb-1">Student Name</div>
                <div className="col-sm-8 fs-5 fw-bold">{certData.fullname}</div>
              </div>

```



```

setIsMinting(true);
setError('');
setResult(null);

try {
  const provider = new ethers.BrowserProvider(window.ethereum);
  const signer = await provider.getSigner();
  const contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI, signer);

  const transactionDetails = `Minting ${formData.crtid} for ${formData.fullname}`;
  const tx = await contract.mintCertificate(
    transactionDetails, formData.sid, formData.fullname, formData.crtid,
    formData.score, formData.grade, formData.coursename, formData.phone, formData.email
  );

  const receipt = await tx.wait();

  const dbResponse = await fetch('http://localhost:5000/api/certificates/save', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      ...formData, transactionHash: receipt.hash, blockHash: receipt.blockHash, contractAddress: CONTRACT_ADDRES
    })
  });

  if (dbResponse.ok) {
    setResult({ message: 'Certificate minted to Blockchain & Database.', transactionHash: receipt.hash });
    setFormData({ crtid: '', sid: '', fullname: '', email: '', phone: '', examid: '', coursename: '', score: '' });
  } else {
    setError('Minted on blockchain, but failed to save to database.');
```

```

        <input type="text" name="crtid" className="form-control form-control-lg bg-light" value={formData.crtid} />
      </div>
      <div className="col-md-4">
        <label className="form-label text-muted fw-semibold">Exam ID</label>
        <input type="text" name="examid" className="form-control form-control-lg bg-light" value={formData.examid} />
      </div>
      <div className="col-md-4">
        <label className="form-label text-muted fw-semibold">Course Name</label>
        <input type="text" name="coursename" className="form-control form-control-lg bg-light" value={formData.coursename} />
      </div>

      <div className="col-12 mt-5">
        <h6 className="text-primary fw-bold text-uppercase tracking-wide border-bottom pb-2 mb-0">Student Information</h6>
      </div>

      <div className="col-md-3">
        <label className="form-label text-muted fw-semibold">Student ID</label>
        <input type="text" name="sid" className="form-control" value={formData.sid} onChange={handleChange} />
      </div>
      <div className="col-md-3">
        <label className="form-label text-muted fw-semibold">Full Name</label>
        <input type="text" name="fullname" className="form-control" value={formData.fullname} onChange={handleChange} />
      </div>
      <div className="col-md-3">
        <label className="form-label text-muted fw-semibold">Email</label>
        <input type="email" name="email" className="form-control" value={formData.email} onChange={handleChange} />
      </div>
      <div className="col-md-3">
        <label className="form-label text-muted fw-semibold">Phone</label>
        <input type="text" name="phone" className="form-control" value={formData.phone} onChange={handleChange} />
      </div>

      <div className="col-12 mt-5">
        <h6 className="text-primary fw-bold text-uppercase tracking-wide border-bottom pb-2 mb-0">Academic Performance</h6>
      </div>

      <div className="col-md-6">
        <label className="form-label text-muted fw-semibold">Total Score</label>
        <input type="text" name="score" className="form-control form-control-lg" placeholder="850/1000" value={formData.score} />
      </div>
      <div className="col-md-6">
        <label className="form-label text-muted fw-semibold">Final Grade</label>
        <select name="grade" className="form-select form-select-lg" value={formData.grade} onChange={handleChange}>
          <option value="">-- Select --</option>
          <option value="A+">A+</option>
          <option value="A">A</option>
          <option value="B+">B+</option>
          <option value="B">B</option>
          <option value="C">C</option>
          <option value="F">Fail</option>
        </select>
      </div>
    </div>
  </div>

  <div className="mt-5 pt-3">
    <button
      type="submit"
      className="btn btn-dark btn-lg w-100 fw-bold shadow-lg rounded-pill py-3"
      disabled={isMinting || !walletAddress}
    >
      {isMinting ? (
        <div>
          <span className="spinner-border spinner-border-sm me-3" role="status" aria-hidden="true"></span>
          Confirming Transaction in MetaMask...
        </div>
      ) : (
        '🚀 Mint Certificate via MetaMask'
      )}
    </button>
  </div>
</form>
</div>
</div>
</div>
</div>
);
};

export default AdminMintingPanel;

```
